

Ingenieria de Software II

Jaime Nazar Anchorena

Contents

Cheatsheet	3
Base de datos	3
NoSQL	3
SQL OLTP	3
SQL OLAP	3
Cache	3
Cassandra	3
MongoDB	3
Objetos	3
Esquemas de base de datos	3
Primario-Secundario	3
Primario-Primario	3
Hosting	4
Cloud	4
On premise	4
Salas cofre	4
Seguridad y conexiones	4
HTTPS	4
JWT	4
WAF	4
VPN	4
Websockets	4
Webhooks	4
Autenticacion	5
2FA	5
MFA	5
Clase 1: Aclaraciones	5
Unidad 1: Introduccion a la Arquitectura de Software	5
Definicion: Ingenieria de Software	5
Definicion: Estilo arquitectonico	5
El rol del arquitecto	6
El cliente	6

Tiene una idea de la solución	6
Tiene un presupuesto	6
Tiene restricciones	6
Planificación previa	6
Qué	7
Cómo	7
Cuándo	7
Calidad	7
Atributos de calidad	8
Armado de la solución	8
ABC - Influencias	8
Stakeholders	9
Negocio	9
Estructura organizacional	9
Experiencia propia	9
Ambiente tecnológico	9
ABC - Fin de ciclo	9
Clase 2: Construcción de Arquitecturas de Software	9
ADD	10
Estilos arquitectónicos	10
Dataflow	10
Batch Secuencial	10
Pipes & Filters	11
Hierarchical Layers	11
Sistemas distribuidos	11
Broker	11
Publish-Subscribe	11
Forwarder-Receiver	11
Client-Dispatcher-Server	12
Sistemas interactivos	12
MVC	12
Sistemas basados en eventos	12
.	12

Cheatsheet

Base de datos

NoSQL

Para una cantidad masiva de datos.

SQL OLTP

Es CA, util para funciones de agregacion.

SQL OLAP

Para uso analitico(reporting), siempre agregando y no eliminando datos.

Cache

Aumenta tiempo de respuestas, pero muy caras.

Cassandra

Columnar, permite cantidades masivas de lectura y escritura. Corre en cluster, mantiene comunicacion entre nodos con Gossip, no sirve cuando se requiere transaccionalidad.

MongoDB

Base de datos NoSQL, no brinda alta disponibilidad. Sirve cuando se requiere flexibilidad y proporciona alto volumen de escrituras.

Objetos

Base de datos NoSQL, guarda objetos complejos en cualquier lenguaje de programacion. Automatiza las operaciones mas comunes(CRUD) y facilita programacion con el tradeoff de performance y complejidad.

Esquemas de base de datos

Primario-Secundario

Busca escalar operaciones de lectura y solo se aplican las escrituras sobre el nodo primario. Si se cae el nodo primario, uno de los secundarios es elegido para tomar su rol.

Primario-Primario

Busca escalar operaciones de escritura, pero trae problemas de concurrencia y por eso no todos los motores RDBM soportan ese modo de operacion. MySQL lo tolera con Group Replication, para Postgres se requieren extensiones. Se sacrifica performance.

Hosting

Cloud

Todo el sistema se encuentra en la nube, como en AWS. Solucion mas barata y permite aumentar el alcance(expandirse a otros paises) de manera eficiente. Cortes de luz/internet ya contemplados.

On premise

Los servidores son propios de la empresa, pueden estar tanto en un datacenter compartido como en el mismo edificio. Aumenta el ownership del sistema, deben contemplarse los casos de cortes de luz/internet.

Salas cofre

Salas para almacenar servidores, similar al anterior.

Seguridad y conexiones

HTTPS

Version cifrada de HTTP

JWT

Firma digital para verificar la identidad del que manda los datos. Util cuando se tiene comunicacion con APIs.

WAF

Servicio de seguridad perimetral que analiza, filtra y bloquea el trafico HTTP/HTTPS malicioso dirigido hacia la webapp. Protege de SQL Injection, XSS, DDoS(solo de capa 7).

VPN

Conexion cifrada y segura mediante un tunel sobre infraestructura publica, permitiendo que los sistemas se comuniquen como si estuvieran en una red privada local.

Websockets

Protocolo de comunicacion bidireccional, full-duplex y persistente sobre unica conexion TCP. Permite una interaccion fluida y en tiempo real entre cliente y servidor sin la sobrecarga del modelo tradicional peticion-respuesta. Traw problemas de availability si se usa con un backend que corre en multiples instancias pues no hay forma de reestablecer la conexion si se cae un servidor del backend.

Webhooks

Mecanismo de comunicacion asincronicos basado en eventos mediando una aplicacion evia automaticamente datos en tiempo real a otra a traves de un HTTP POST. No se puede usar con una SPA.

Autenticacion

2FA

Subconjunto de MFA, exige dos evidencias de distinta categoria para validar la identidad de un usuario.

MFA

Mas general, requiere dos o mas factores de verificacion independiente de distinta categoria.

Clase 1: Aclaraciones

- Es diferente dependiendo de la comision
- Es totalmente practica
- Hacer y consultar por mail todo lo posible
- Mediados de octubre unico parcial
- El final es teorico/practico, IMP mandar mail de como esta formado el grupo
- No se manda el enunciado del final hasta que todos manden el mail
- El cuadro de atributos es el unico que se puede tener en el examen
-

Unidad 1: Introduccion a la Arquitectura de Software

Definicion: Ingenieria de Software

La Ingeniería de Software es la rama de la ingeniería que aplica los principios de la ciencia de la computación y las matemáticas para lograr soluciones costo-efectivas a los problemas de desarrollo de software, es decir, permite elaborar consistentemente productos correctos y costo-efectivos.

Contruir un proyecto de software es muy complejos, ocupa tareas distintas y por tanto requiere de perfiles distintos(que podrian o no ser de la misma persona).

Software architecture encompasses the set of significant decisions about the organization of a software system including: - The selection of the structural elements and their interfaces, - Behavior as specified in collaboration among those elements, - An architectural style that guides this organization.”

Definicion: Estilo arquitectonico

Igual que con patrones, el **estilo arquitectonico** tambien se toma de la arquitectura. Por ejemplo, decir que un edificio es brutalista o gotico nos dice un monton de que se puede encontrar adentro, cuando se contruyo, etc. La totalidad del edificio sigue estos lineamientos, y esa es la idea que llevamos al software. Nos indica como funciona nuestro sistema como un todo.

La coherencia interna es importante y no se puede cambiar, sobrevivira el paso del tiempo.

Un sistema legacy es un sistema sumamente existoso pues perduro a pesar del paso del tiempo. Se sigue manteniendolo pues aun aporta valor.

El rol del arquitecto

Su definicion varia segun la empresa, pero son tren tipos:

- Arquitecto de soluciones(rol de negocio)
- Arquitecto de integracion(rol especifico)
- Arquitecto de software(rol operativo, el mas generico)

Vamos a tratar especialmente del arquitecto de soluciones, y un poco del de integracion en SOA. Actualmente, el rol del arquitecto esta desapareciendo, en la practica cada uno del equipo de programacion esta creando una arquitectura de software sin la necesidad de tener el rol. Sin embargo, para ello se requiere:

- **Liderazgo:** Debe ser el modelo a seguir.
- **Negocio:** No debe perder de vista que el proyecto soluciona un problema.
- **Técnico:** Desde programación a hardware.
- **Comunicación:** Debe convencer a jefes, equipo, cliente, etc.
- **Negociación:** Siempre hay compromisos.
- **Visión:** Las decisiones son hacia el futuro.

Es importante conocer el negocio, la necesidad presente y futura sobre la que tiene que operar. Hay que predecir la direccion que tomara el negocio para tener en cuenta futuros cambios.

El cliente

Al final del dia hay un problema, ya sea interno o externo de la organizacion, para el cual se necesitauna solucion. El cliente tienen una idea general y el grado de detalle suele variar mucho. Sabe lo que quiere conseguir, pero no sabe la complejidad y el costo que tiene. El cliente:

Tiene una idea de la solución

- Sabe qué quiere (a grandes rasgos)
- IWKWYSI (I will know when I see it)
- Ignora la complejidad real de la tarea

Tiene un presupuesto

- Cuanto menos, mejor

Incluso si dice que no, tiene un presupuesto y se va a bsucar lo mos barato.

Tiene restricciones

- Fecha de puesta en producción
- Tecnológicas
- Regulaciones

Planificacion previa

Se necesita **mucha** planificacion previa.

Qué

- Alcance
- Factibilidad
- Riesgos

Son los requerimientos, los de usuario(AKA casos de uso), los del sistema, los funcionales y los no funcionales. Los no funcionales no son autocontenidos, son transversales al sistema y no tienen principio ni fin, a diferencia de los funcionales(no pueden ser un user story).

Funcionales Los funcionales son los fáciles, tanto de programar como de validar. Nos vamos a concentrar en los no funcionales pues ya sabemos como hacerlos. Sin embargo, son difíciles de relevar, como se si son todos? como se si son los correctos?

Se puede hablar con el cliente, pero como sabemos si el cliente los sabe. Es la **incertidumbre**, inherente a cualquier proyecto de software, y solo desaparece cuando el proyecto se termina.

Nunca se va a poder eliminar la incertidumbre, pero hay que empezar eventualmente. Las cosas que no quedaron claras se resuelven con supuestos, los cuales deben ser:

- Posibles
- Probables

No hay forma de que la arquitectura no falle, no es improbable

Siempre deben ser documentados, al final del día es la definición misma del sistema.

Cómo

- Diseño del sistema
- Recursos de hardware y software requeridos
- Equipo necesario

Cuándo

- Armar un plan de trabajo, estimar y planificar

Esto permite además conocer el costo. El costo mas grande en un proyecto de software son los desarrolladores.

Calidad

Define si el sistema cubre todos los casos de uso, pero no alcanza con si los cumple, sino que tambien debe tener en cuenta como.

The capability of a software product to satisfy stated and implied needs when used under specified conditions(IEEE)

La calidad tiene multiples dimensiones, las cuales se conocen como atributos de calidad. El ISO 25000 tiene una(no la unica) definicion de atributos de calidad, son muchos.

Atributos de calidad

Todos son criticos para el exito, se infieren a partir de los requerimientos no funcionales. Es importante determinar cuales son los que tenemos que poner el foco.

En ingles, como son habilidades, terminan en -ity

No vamos a abarcar todos, sino los mas comunes(ver cuadro en apunte). Al momento de determinarlos se infieren de:

- La operación del negocio
- El marco regulatorio
- El entorno competitivo y las expectativas del usuario

Reliability es mas duro que disponibilidad, pues se refiere a cuanto el sistema pueda perdurar en el tiempo y funcionar de manera correcta

Testability no es tan trivial, hay sistemas que son dificiles de probar. Por ejemplo, el envio de mails en PAW

Importante destacar, que no todos los atributos son **criticos**, hay que elegir cuales son los que realmente importa, no significa que se ignoren los demas, sino que no se pondra ningun esfuerzo ecepcional para complirlos.

Por otro lado, hay que cuantificarlos a los que elegimos, un sistema que necesite ser disponible, cuan disponible? 90%, 99%?

No existe 100% de disponibilidad, cada 9 que le sumo incrimienta el costo en un orden de magnitud

Entonces, hay que poner un limite de cuanto se quiere perfeccionar cada atributo, lo cual en si es otra decision que asume riesgos. Este tipo de desiciones debe poder ser justificadas y demostrar que se cumple. La mayor fuerza suele ser el marco regulatorio, de que se puede esperar como minimo.

Armado de la solucion

El Arquitecto debe tomar todos los requerimientos y constraints y catalizarlos en una solución. Todo esto, que no es poca cosa, no es todo lo que define una arquitectura, la arquitectura tambien es resultado de las influencias del arquitecto, de lo disponible tecnologicamente, de las buenas practicas del momento, la experiencia de la organizacion y del arquitecto(son factores internos y externos).

Pero estos no son los únicos inputs que definen la arquitectura... La arquitectura es el resultado de influencias en aspectos técnicos, sociales, culturales y de negocio.

Una arquitectura influye en aspectos técnicos, sociales, culturales y de negocio de futuras arquitecturas.

Esto es el Architecture-Business Cycle (ABC)

ABC - Influencias

Esto tiene sus consecuencias, pues el equipo mejora sus habilidades con las tecnologias, la organizacion tiene mas sistemas a mantener, etc. Esto genera un loop de auto-refuerzo donde sera mas propensa la empresa a volver a usar esas tecnologias.

Stakeholders

Tienen expectativas y restricciones sobre el sistema, como presupuestarias, donde a veces se pueden contradecir entre si. El arquitecto debe detectarlas todas y buscar el mejor compromiso posible.

Negocio

Hay dos tipos a tener en cuenta:

- Negocio inmediato(Aprovecha sistemas existentes)
- Negocio a largo plazo(Invertir en nueva infraestructura para el futuro)

Estructura organizacional

Los conocimientos previos, que licencias hay disponibles, que herramientas hay, como trabaja el equipo.

Experiencia propia

Es raro que se vuelva a utilizar una tecnologia que dio resultados pobres en el pasado.

Ambiente tecnologico

La tecnología disponible, estándares de la industria o técnicas de ingeniería de software influye sobre ella.

Al final del dia el objetivo fundamental es reducir el riesgo

Cuando una tecnologia que ya usamos falla es mas facil de resolver que con una tecnologia nueva, y por lo general los fallos son en produccion. Esto plantea una paradoja, pues se quiere reducir los riesgos a la vez que se quiere mantener reelevante en el mercado.

ABC - Fin de ciclo

Una vez se logra implementar una tecnologia nueva, se convierte en una decision estrategica, fue un riesgo. Por lo tanto, hay que ser cauto sobre que proyecto se utiliza para probar algo nuevo.

Encima, una vez funcione el sistema, hay que mantenerlo, se debe seguir utilizandolo. Hay que decidir cuando se va a diversificar el stack tecnologico.

Una vez que la arquitectura está creada y el sistema ha sido desarrollado, ambos afectarán:

- La estructura y objetivos de la organización que los desarrolló
- Los requerimientos futuros de los clientes
- La experiencia de los arquitectos y desarrolladores, dado que la base de experiencia corporativa ha sido mejorada
- Generan tecnología que será tenida en cuenta para ser aprovechada en futuros proyectos.

Es un problema de gestion de riesgo

Clase 2: Construcción de Arquitecturas de Software

De los requerimientos funcionales y de los atributos de calidad se debe producir una arquitectura candidata. Es una forma de diseño llamada Attribute Driven Design. Una vez se tiene una arquitectura candidata se

lleva a cabo un proceso de iteracion para perfeccionarla.

Con ADD se tiene como entrada General escenarios que se utilizan para formar Requirements. Luego se llega a una arquitectura.

ADD

0. Se da un orden de prioridad a los atributos de calidad, pues mejorar una atenta en contra de otra. Esto implica la creacion de un criterio para saber cual priorizar.
1. Determinar como va a ser la arquitectura, conceptualizarla. Usuarios, sistemas externos, donde va a ir la informacion, etc. Version mas inocente y minimalista que cumpla con los requerimientos funcionales.
2. Tomar el primer atributo y proponer escenarios que los pongan a prueba, esto permite identificar donde se rompe, los riesgos. Estos escenarios se conocen como los **drivers de arquitectura**. Hay riesgos que se pueden determinar como aceptables, aceptando que pueden pasar cosas, entonces hay que determinar como solucionar el fallo.
3. Si se cambia algo hay que re-plantearse cada escenario, volver al paso 2. Entonces, cuantos mas escenarios hay, mas se tarda en volver a analizar.
4. Fin, se obtiene una arquitectura candidata, encenarios, riesgos asumidos, trade-offs asumidos y supuestos asumidos. Sirven para la verificacion post implementacion, para que lo verifique un stakeholder y simplemente como documentacion.

Los primeros escenarios del paso 2 son los mas probables

Por mas que haya riesgos que se asumen, hay que documentarlos y crear un plan de contingencia.

Estilos arquitectonicos

Es lo que se busca al conceptualizar el problema, son patrones de alto nivel que describen la organizacion general de un sistema en pos de ciertos atributos de calidad. Seguir un estilo ya probado provee consistencia y previsibilidad.

Dataflow

Estilo arquitectonico, externosl objetivo es procesamiento de datos, no necesariamente un sistema interactivo. Hay varios:

Batch Secuencial

Los datos se transmiten entre programas como un todo, en general mediante archivo. El procesamiento ocurre offline, no hay nada sincronico y los procesos no saben de la existencia de los demas. Se acumula informacion y en un momento dado se procesa.

Pero, si falla, se tiene que revertir todo y abortar el proceso o por ahi puede guardar el punto de procesamiento y resumir luego. Hay que documentar que hara en caso de un error.

Este estilo tiene como ventaja procesos independientes y permiten el reprocesamiento sin perdida de informacion. Sin embargo, no permite procesamiento real-time. En todo los casos documentar como manejar fallas.

Pipes & Filters

En este caso se tiene procesamiento en tiempo real, se tienen procesos chicos donde mediante pipes la salida de uno es la entrada del siguiente. Es un stream de datos, es decir, no se necesita de un fin para poder emitir la salida. Son procesos extremadamente flexibles y se utilizan mucho en los OS. Pero bueno, si alguno de los procesos falla, cualquier dato que este en tránsito se pierde. Entonces, tiene una baja tolerancia a errores en el procesamiento. Es el pipeline clásico de arquitecturas.

Hierarchical Layers

Cada capa provee servicios a la capa superior y es cliente de la capa inferior, el problema es encontrar el número perfecto de capas. Las capas pueden ser lógicas o físicas. Divide un problema complejo en abstracciones, mejoras en una capa impactan en todo el sistema y suponen reusabilidad y fácil intercambio de implementaciones.

Sin embargo, no todos los sistemas se pueden estructurar así, hay un posible deterioro de performance con muchas capas (peor si son físicas, por ejemplo, saltos entre servidores) y los cambios suelen tener que replicarse en todas las capas.

Sistemas distribuidos

Como se descubren miembros de la red y como se comunican.

Broker

Coordina comunicación entre clientes y servidores, busca resolver el problema de encontrar un servicio y el de como comunicarse con el servicio. Lo hace operando como un proxy, los servidores exponen servicio a partir del broker. Facilita el discovery de servicios, los servicios pueden no estar expuestos en forma directa a la red, pero el broker es un Single Point Of Failure (SPOF) stateful. Si falla se cae todo.

Publish-Subscribe

Análogo al patrón **Observer** en un contexto distribuido. Un componente **publisher** notifica a muchos consumidores **subscribers** suscritos a él. Hay un sistema en el medio que se encarga de hacer el broadcast a los subscribers, se conoce como el **topic** (tema). Este se encargará de hacer la multiplexación. Además, están completamente desacoplados, el sistema de origen no sabe de la existencia de los receptores y viceversa.

Entonces, los publishers y subscribers no se conocen mutuamente, solo se conocen los topics. Es posible agregar y quitar suscriptores en forma transparente. La desventaja que tiene es que requiere consideraciones especiales respecto de la pérdida de eventos, según que solución elija puede o no haber pérdida de mensajes.

Un webhook es muy parecido, hay implementaciones de topics que funcionan como webhooks. Pero difieren en como lo implementan.

Forwarder-Receiver

Resuelve exclusivamente los problemas de comunicación, se abstraen por completo y resuelven solo los problemas. Permite consumir servicios remotos como si fuesen métodos locales. Corba y Java RMI son implementaciones. Entonces, simplifica el uso al ser transparente, pero se esconde la complejidad, por lo que se desconoce si tiene mucho overhead, si hay algún problema oculto, etc.

Client-Dispatcher-Server

Resuelve el problema de encontrar a los servicios, pero la conexión va a ser directa. Permite conexiones cuando el **server** no tiene una dirección fija. Por ejemplo, bit torrent, aplicaciones de mensajería / videoconferencia.

Sistemas interactivos

Sistemas que esperan a un input, para poder reflejar el cambio en pantalla. Clásico, MVC, PAC, MVP, VIPER, etc. Conceptualmente, todas estas opciones hacen lo mismo, lo que cambia son las abstracciones y responsabilidades. El objetivo es separar la lógica de negocio de las vistas de presentación.

MVC

El más popular, divide la aplicación en Modelos, Vistas y Controladores. La desventaja principal es que hay un **trickle effect** entre componentes cuando cambia el modelo de datos.

Sistemas basados en eventos

Permite crear sistemas distribuidos con bajo acoplamiento. El sistema se divide en componentes completamente agnósticos uno de otros. Conceptualmente existen tres componentes:

- Event producer
- Channel
- Event consumer

Notar que es conceptual pues un componente puede ser producir y consumer al mismo tiempo. Estilos aplicables:

- Single Event Processing
- Event Stream Processing
- Complex Event Processing

- Online Event Processing

Complex event: Si un usuario hace un login y cambia password y va a hacer una compra a un artículo muy caro, son eventos normales, pero si pasan las tres en secuencia es un indicador de posible fraude. Este tipo de análisis es lo que permite hacer este estilo.

Tiene ventajas y desventajas(ver apuntes).

Existen nun monton mas de estilos, investigar a voluntad.

Libro para investigar patrones: Patterns of Enterprise Application architecture.